

Virtual Power Plant as a Service (VPPaaS)

Enterprise Integration: Sprint 2 Technical Report Business Process Orchestration with Camunda 8 and Kong API Gateway

Afonso Carvalho (*IST 116482*)
Eduardo Silva (*IST 106929*)

June 2026

Abstract

This report documents the Sprint 2 deliverable of the VPPaaS Enterprise Integration project. While Sprint 1 delivered the data plane (microservices, shared RDS database, Kafka backbone), Sprint 2 *orchestrates* it into the eight business processes of the project brief using **Camunda 8** as the process engine and **Kong API Gateway** as the single entry point to every microservice. It covers (i) how each of the eight flows was modelled in BPMN and wired to the microservices through Kong; (ii) their end-to-end execution against the live AWS deployment, including the defects found and corrected during testing; and (iii) the documentation of the source code, Terraform infrastructure-as-code, installation procedures and parametrizations of the integration layer.

Contents

1	Introduction	2
2	Business Process Architecture	2
2.1	Integration platform: Camunda 8 and Kong API Gateway	2
2.2	Modelling pattern: Initiator/Executor collaborations	3
2.3	Mapping the eight required business processes to BPMN models	3
2.4	Detailed description of each flow	4
3	End-to-end Testing	5
3.1	Test environment and methodology	5
3.2	Full end-to-end execution: Prosumer Management	6
3.3	Defects identified and corrected	7
3.4	Test procedures for the remaining seven flows	8
3.5	Automated end-to-end test suite (e2e-tests/)	8
4	Source Code, Infrastructure and Parametrization Documentation	9
4.1	Repository layout of the orchestration layer	9
4.2	Camunda 8 engine: Terraform	9
4.3	Kong API Gateway: Infrastructure, Registration and Routing	10
4.4	Konga administration UI	11
4.5	BPMN process definitions: parametrization conventions	11
5	Conclusion	11

1 Introduction

The VPPaaS platform aggregates Distributed Energy Resources (DER), namely battery storage systems (BESS), solar inverters (PV) and EV chargers, and exposes them as a single virtual power plant that a Utility Operator can use to balance supply and demand across grid cells. Sprint 1 delivered the data plane (the domain microservices, the shared AWS RDS database and the Kafka event backbone, all provisioned through Terraform), documented in Sprint 1.

Sprint 2 closes the loop required by the project brief: *“orchestrate the business processes that coordinate these microservices, expose them through an API Gateway, and demonstrate that the resulting flows execute correctly from end to end”*. Concretely, the brief requires:

- (1) Modelling and deploying, through **Camunda** (orchestration engine) and **Kong** (API Gateway), the eight business processes that the platform must support: *Prosumer Management, Utility Operator Management, Asset Link Management, Telemetry Ingestion, Flexibility Emission, Grid Balancing Recommendation, Energy Analytics and Flexibility Forecasting (AI)*;
- (2) Documenting **end-to-end tests** of the full execution of these business processes against the deployed system;
- (3) Documenting the **source code, Terraform scripts, BPMN models, installation procedures and parametrizations** that make the integration layer reproducible.

Sections 2–4 below address these three points in order.

2 Business Process Architecture

2.1 Integration platform: Camunda 8 and Kong API Gateway

Two new infrastructure components were added on top of the Sprint 1 data plane:

- **Camunda 8 (Zeebe + Operate + Tasklist), EC2 t3.large**: the orchestration engine that executes the BPMN process definitions, exposes the `/v2/process-instances` and `/v2/deployments` REST APIs, renders generated forms in Tasklist and gives full execution visibility (active instances, completed/incident instances, variables) in Operate. Provisioned by `Camunda-Terraform/EC2CamundaEngine.tf`, an `aws_instance` of type `t3.large` with an all-open security group (ports 8080/8081/8082), running the official Camunda 8.8.9 Docker Compose stack via the `deploy.sh` user-data script. Login: `demo/demo`.
- **Kong API Gateway, EC2 t3.small**: the single public entry point for *every* microservice. Provisioned by `KongTerraform/EC2Kong.tf` and configured at boot by `setup-kongGateway.sh`, which registers one Kong *service* and one *route* per microservice (see Table 1). Every BPMN service task that needs to reach a microservice calls Kong, never the microservice’s EC2 address directly, which is exactly the integration pattern the brief asks for (“expose the business processes through an API Gateway”).
- **Konga, EC2 t3.small**: a web administration UI for Kong (`KongaTerraform/EC2Konga.tf`), used during development to inspect/validate the services and routes that `setup-kongGateway.sh` registers.

With `strip_path=true`, a request such as `GET http://<kong-dns>:8000/prosumer/Prosumer` arrives at the Prosumer microservice as `GET /Prosumer`. All the BPMN flows described below use this single Kong base URL as the target of their HTTP Connector service tasks, so the orchestration layer never needs to know individual microservice EC2 addresses.

Table 1: Kong services and routes registered by `setup-kongGateway.sh` (one per microservice)

Kong service name	Route path (strip_path=true)	Upstream microservice
prosumer-service	/prosumer	Prosumer MS
utility-operator-service	/utility-operator	UtilityOperator MS
assetlink-service	/assetlink	AssetLink MS
telemetry-service	/telemetry	Telemetry MS
flexibility-event-service	/flexibility-event	FlexibilityEvent MS
grid-balancing-service	/grid-balancing	GridBalancing MS
energy-analytics-service	/energy-analytics	EnergyAnalytics MS
artificial-intelligence-service	/artificial-intelligence	ArtificialIntelligence MS

2.2 Modelling pattern: Initiator/Executor collaborations

Two of the eight flows (*Prosumer Management* and *Asset Link Management/Telemetry Ingestion*) were modelled as **BPMN collaborations** made up of several independently-deployable processes that communicate through asynchronous *messages*, following a classic “Initiator → Executor” pattern:

1. The *Initiator* process starts from a **none start event**, collects input through a Tasklist user-task form (e.g. the data of the Prosumer to create), and then **publishes a message** that starts a second, independent process instance (the *Executor*).
2. The *Executor* process is triggered by a **message start event**, runs the actual business logic (promise/possibility checks, the REST call to the microservice through Kong, and the declaration step) and finally publishes a confirmation message back to the Initiator.
3. The Initiator resumes on a matching **message catch event** and completes with an acceptance user task.

This mirrors the request/promise/declare/accept commitment lifecycle described in the project brief’s business-architecture section, and demonstrates asynchronous, message-based collaboration between independently deployed process definitions, rather than just a single linear flow. The remaining flows (Utility Operator Management, Telemetry/AI/Analytics/Flexibility/Balancing) are modelled as single linear processes, since their business logic does not require cross-participant negotiation.

A structural detail that is easy to overlook (and that produced one of the defects documented in Section 3.3) is that **every variable the Executor needs (in particular `camundaBaseUrl`, used by Tasklist form-generation service tasks) must be explicitly forwarded in the variables object of the message that starts it**; Camunda does not automatically share process variables across independent process instances.

2.3 Mapping the eight required business processes to BPMN models

Table 2 maps each of the eight business processes required by the brief to its BPMN file, Camunda `processDefinitionId(s)` and the microservices it orchestrates (always reached through Kong, per Table 1).

Note that *Asset Link Management* and *Telemetry Ingestion* are modelled inside the **same** BPMN file (`AssetLinkManagement.bpmn`) as a three-participant collaboration (`ProsumerForAssetLink` → `AssetLinkManagement` → `TelemetryManagement`); this reflects the real dependency described in the brief, where activating an asset link is what triggers the creation of the dynamic Kafka consumer for that link’s telemetry topic.

Table 2: The eight required business processes and their BPMN realisation

Business process	BPMN file	Microservices orchestrated
1. Prosumer Management	ProsumerManagement.bpmn	Prosumer MS
2. Utility Operator Management	UtilityOperatorManagement.bpmn	UtilityOperator MS
3. Asset Link Management	AssetLinkManagement.bpmn	Prosumer MS, AssetLink MS
4. Telemetry Ingestion	AssetLinkManagement.bpmn (3rd pool)	AssetLink MS, Telemetry MS
5. Flexibility Emission	FlexibilityEmission.bpmn	Telemetry MS, FlexibilityEvent MS
6. Grid Balancing	GridBalancing.bpmn	UtilityOperator MS, Telemetry MS, Prosumer MS, GridBalancing MS
7. Energy Analytics	EnergyAnalytics.bpmn	Telemetry MS, Prosumer MS, UtilityOperator MS, EnergyAnalytics MS
8. Flexibility Forecasting (AI)	AIForecasting.bpmn	FlexibilityEvent MS, ArtificialIntelligence MS

2.4 Detailed description of each flow

1. Prosumer Management

Collaboration ProsumerForAssetLink-style Initiator/Executor. The Initiator process ProsumerMngInitiator starts from a none start event and presents the user task “*Decide the Data for Prosumer Creation order*” (generated form fields: Name, Fiscal Number, Location). The service task “*Request Prosumer Creation order*” then publishes a message carrying name, location, fiscalnumber and camundaBaseUrl, starting the Executor process. The Executor runs “*Verify if execute is possible*” (a checkbox user task: “*Is it possible to promise the request?*”), “*Promise Prosumer Creation order*”, the HTTP Connector service task “*Create Prosumer*” (POST /prosumer/Prosumer via Kong), “*Declare Prosumer Creation order*” and finally “*Accept Prosumer Creation order*”, after which both processes reach their end events. This is the flow that was driven to completion in the live environment; see Section 3.2.

2. Utility Operator Management

Single linear process UtilityOperatorManagement, with no required start variables. It chains seven user/service tasks, each assignee=demo: *Request* → *Verify if ... is possible* → *Promise* → *Create Utility Operator* (real REST call, POST /utility-operator/UtilityOperator (via Kong)) → *Declare* → *Check* → *Accept*, each gated by an “*Is it possible/acceptable?*” checkbox form. Input data collected on the first task: Name, Location.

3. Asset Link Management

Three-participant collaboration inside AssetLinkManagement.bpmn: ProsumerForAssetLink (Initiator, none start event) → AssetLinkManagement (message start, messageRef = Message_2r1j62e) → TelemetryManagement (message start, messageRef = Message_2mp8beu). The Initiator’s user task “*Decide the data to AssetLink association order*” (form FormChoiceDataForAssetLink; fields UtilityOperatorID, prosumerID) feeds a message that starts AssetLinkManagement, which runs “*Verify if execute product is possible*” (form Form_Promise_AssetLinkMng, checkbox promise) and the REST call that creates the AssetLink record (via Kong, /assetlink/AssetLink); on completion it both replies to the Initiator and starts TelemetryManagement. The Initiator finally collects the acceptance user tasks “*Check AssetLink association order*” / “*Check Telemetry Consumer Order*” (form Form_Accept_AssetLinkMng, checkbox accept).

4. Telemetry Ingestion

Third participant of the same collaboration, process TelemetryManagement. Triggered by the message published by **AssetLinkManagement** once an asset link is created, it runs its own “*Verify if execute product is possible*” step (same form/checkbox pattern, **Form_Promise_AssetLinkMng/promise**) and the HTTP Connector service task that calls **POST /telemetry/Telemetry/Consume** (via Kong) to start the dynamic Kafka consumer for the new {**assetLinkId**}-{**utilityOperatorId**} topic, the integration point between the orchestration layer and the Telemetry MS’s **DynamicTopicConsumer** described in Sprint 1. It then replies with a confirmation message consumed back in the Initiator.

5. Flexibility Emission

Single linear process FlexibilityEmissionProcess. The user task “*Select Asset for Flexibility Emission*” collects **Asset ID** (e.g. **BATT-001/SOLAR-001**) and **Grid Cell ID** (e.g. **LISBON-DT**); automatic service tasks then query the latest telemetry for that asset through Kong (**/telemetry/Telemetry/asset/...**) and call **POST .../flexibility-event/Flexibility/Emit**, exercising the rule-based emission logic of the **FlexibilityEvent MS**.

6. Grid Balancing Recommendation

Single linear process GridBalancingProcess. The user task “*Select Grid Zone for Balancing*” collects **Grid Cell ID** (**LISBON-DT**); the process then automatically aggregates data from the **UtilityOperator**, **Telemetry**, **AssetLink** and **Prosumer** microservices (through Kong) and calls **POST .../grid-balancing/GridBalancing/Emit**.

7. Energy Analytics

Single linear process EnergyAnalyticsProcess. The user task “*Select Zone for Energy Analytics*” collects **Grid Cell ID**; the process aggregates telemetry, prosumer and operator data through Kong and calls **POST .../energy-analytics/EnergyAnalytics/Publish**.

8. Flexibility Forecasting (AI)

Single linear process AIForecastingProcess. The user task “*Select Zone for AI Flexibility Forecast*” collects **Grid Cell ID** (**LISBON-DT**); the automatic chain “*Get Flexibility Event History*” → “*AI Analysis via Ollama (ArtificialIntelligence MS)*” → “*Emit AI Flexibility Event (AI_DISCHARGE)*” The chain it queries historical **FlexibilityEvent** rows, sends them to the **ArtificialIntelligence MS** (**Ollama / llama3.2**) through Kong, and republishes the AI-enriched event with **offerType = AI_DISCHARGE**. As this performs a real LLM inference call, its execution time is noticeably longer than the other flows.

3 End-to-end Testing

3.1 Test environment and methodology

All tests were executed against the live AWS deployment produced by **DeploymentAutomation-ubuntu.sh / -macOS.sh** (Sprint 1 infrastructure plus the Camunda, Kong and Konga instances added for Sprint 2):

- **Camunda 8:** **http://<camunda-dns>:8080** (Operate at **/operate**, login **demo/demo**);
- **Kong Gateway:** **http://<kong-dns>:8000**;
- **Konga:** **http://<konga-dns>:1337**.

The placeholders `<camunda-dns>`, `<kong-dns>` and `<konga-dns>` stand for the public DNS names of the respective EC2 instances, which change on every redeploy: they are printed at the end of `DeploymentAutomation-ubuntu.sh` and `-macOS.sh` and obtainable via `terraform` output in `Camunda-Terraform/`, `KongTerraform/` and `KongaTerraform/` respectively (see also the placeholder table in `BPMN/EndToEnd_Tests.md` for the values used in the run described below).

Method. Every flow with a `none` start event is started with a `POST /v2/process-instances` call against the Camunda REST API (see Listing 1 for the generic template); the execution is then driven step by step in **Operate**, completing the generated Tasklist forms with realistic seed data (the LISBON-DT grid cell, the ArcoCegoLisbon operator, the seed prosumer `client1` and assets `BATT-001/SOLAR-001` created in Sprint 1, plus the new João Silva prosumer created during this sprint’s tests). Completion is verified by querying the corresponding microservice **through Kong** and checking that a new row was persisted in the shared RDS database. The full script of commands and form values for all eight flows is kept in `BPMN/EndToEnd_Tests.md`, written specifically to support the live defence demonstration.

```
1 curl -X POST "http://<camunda-dns>:8080/v2/process-instances" \ -H "Content-Type: application/json" \ -d '{"processDefinitionId": "<PROCESS_ID>", "variables": { ... } }'
```

Listing 1: Generic template for starting any none-start-event process

3.2 Full end-to-end execution: Prosumer Management

The *Prosumer Management* flow was driven to completion end to end: from the Camunda REST API, through every user task in Operate/Tasklist, through Kong, into the Prosumer microservice, and finally into a persisted row in the shared RDS MySQL database. This is the flow chosen to demonstrate, in the defence, that the orchestration layer genuinely calls the deployed microservices (rather than stopping at a mocked or partially-wired model).

1. Start the Initiator instance:

```
1 curl -X POST "http://<camunda-dns>:8080/v2/process-instances" \ -H "Content-Type: application/json" \ -d '{"processDefinitionId": "ProsumerMngInitiator", "variables": {"camundaBaseUrl": "http://<camunda-dns>:8080"} }'
```

2. Drive the flow in Operate/Tasklist (Section 2 describes each step in detail): fill “*Decide the Data for Prosumer Creation order*” with Name = João Silva, Fiscal Number = 123456789, Location = Lisboa; the Executor process starts automatically; complete “*Verify if execute is possible*” by checking “*Is it possible to promise the request?*” = yes; the chain *Promise* → *Create Prosumer* → *Declare* → *Accept* runs automatically and both process instances reach their end events.

3. Verify persistence through Kong:

```
1 $ curl http://<kong-dns>:8000/prosumer/Prosumer
2 [ {"FiscalNumber": 123456, "id": 1, "location": "Lisbon", "name": "client1"}, {"FiscalNumber": 123456789, "id": 2, "location": "Lisboa", "name": "Joao Silva"}
3 ]
```

Listing 2: Verification call and observed result: a new row was created

Record `id = 2` is the row created live by the orchestrated flow, confirming that the chain **Camunda** → **Kong** → **Prosumer microservice** → **RDS MySQL** works end to end. (An earlier run produced a row with `FiscalNumber: null`; that defect, and how it was corrected, is documented as Defect 3 in Section 3.3, and the run shown above is the corrected, fresh execution.)

3.3 Defects identified and corrected

Driving the Prosumer flow end to end, and then proactively re-checking the remaining seven flows for the same classes of problem, surfaced three systemic defects in the BPMN models. None of them were visible from a static read of the diagrams; all three only manifested at runtime, which is precisely why end-to-end testing (rather than design-time validation alone) is required by the brief. All three were fixed *at the BPMN-definition level* and the affected processes were **redeployed and re-run from a clean instance** (the user’s explicit preference over patching a running incident through Operate’s variable panel, since a definition-level bug will simply recur on the next instantiation if only the symptom is patched).

Defect	Symptom & root cause	Fix applied
1. <code>camundaBaseUrl</code> not propagated between collaborating processes	Incident raised in Operate: <code>jakarta.validation.ValidationException: Property: url: ... must not be blank</code> . Root cause: the Initiator never forwarded its <code>camundaBaseUrl</code> variable inside the message body that starts the Executor, so the Executor’s Tasklist form-generation service task received a blank URL. This is a <i>structural</i> collaboration bug, not a one-off.	Added <code>"camundaBaseUrl": camundaBaseUrl</code> as the first key of the <code>variables</code> object published by the message-sending service tasks: in <code>ProsumerManagement.bpmn</code> (“Request Prosumer Creation order”, Initiator → Executor) and, proactively, in <i>both</i> message-publication points of <code>AssetLinkManagement.bpmn</code> (<code>ProsumerForAssetLink</code> → <code>AssetLinkManagement</code> and → <code>TelemetryManagement</code>).
2. Empty Kong-address placeholder in service-task URLs	Service tasks failed with a malformed target <code>http://:8000/prosumer/Prosumer</code> (empty host). Root cause: an unsubstituted <code>KONG_ADDRESS</code> placeholder left over from the BPMN authoring/templating step, present not just in Prosumer Management but systemically across the project.	Found 21 occurrences across all 7 BPMN files (1 in <code>ProsumerManagement</code> , fixed individually first; 20 more in <code>AIForecasting</code> , <code>AssetLinkManagement</code> , <code>EnergyAnalytics</code> , <code>UtilityOperatorManagement</code> , <code>GridBalancing</code> , <code>FlexibilityEmission</code>). Corrected with one global substitution (Listing 3) replacing the empty host with the live Kong DNS, verified afterwards with a zero-result <code>grep</code> for the broken pattern.
3. Variable name case-mismatch dropped the fiscal number	The created Prosumer record persisted with <code>"FiscalNumber": null</code> . Root cause: the “Create Prosumer” service task referenced the FEEL variable <code>fiscalNumber</code> (camelCase), while the actual process variable produced by the form is <code>fiscalnumber</code> (lowercase), so the expression resolved to <code>null</code> .	Corrected the request-body expression in <code>ProsumerManagement.bpmn</code> from <code>"FiscalNumber": fiscalNumber</code> to <code>"FiscalNumber": fiscalnumber</code> ; redeployed and re-ran a fresh instance, which produced the correctly-populated record <code>id = 2</code> shown in Listing 5 (<code>FiscalNumber = 123456789</code>).

Table 3: Defects found while end-to-end testing the BPMN flows, their root cause and the corrective action taken at the process-definition level.

```
1 cd BPMN && sed -i \ 's|http://:8000|http://<kong-dns>:8000|g' \
  AIForecasting.bpmn AssetLinkManagement.bpmn EnergyAnalytics.bpmn \
  UtilityOperatorManagement.bpmn GridBalancing.bpmn FlexibilityEmission.
  bpmn
```

Listing 3: Single global substitution that corrected the empty-Kong-address defect (Defect 2) in 20 locations across 6 files in one pass

After every edit to a `.bpmn` file, the corrected definition must be **redeployed** before being tested again (Listing 4); a running instance keeps executing the version of the definition it was started with, so an in-place edit has no effect on instances already in flight.

```
1 curl -L -X POST "http://<camunda-dns>:8080/v2/deployments" \ -H "Content-Type: multipart/form-data" -H "Accept: application/json" \ -F "resources=@./BPMN/<FILE_NAME>.bpmn"
```

Listing 4: Redeploying a corrected BPMN definition to Camunda

```
1 {"FiscalNumber": 123456789, "id": 2, "location": "Lisboa", "name": "Joao Silva"}
```

Listing 5: Final verification result referenced from Section 3.2 (Defect 3 corrected: FiscalNumber persists)

3.4 Test procedures for the remaining seven flows

To make the remaining flows reproducible for the live defence, every flow was documented with its exact start command, its Tasklist step sequence (with concrete seed-data values to type into each form) and its Kong verification call, in `BPMN/EndToEnd_Tests.md`. The flows are grouped by modelling complexity:

- **Group 1: single-process flows** (*Utility Operator Management, AI Forecasting, Energy Analytics, Flexibility Emission, Grid Balancing Recommendation*): no collaboration/messages, no `camundaBaseUrl` requirement; only affected by Defect 2 (broken Kong address), already corrected in all six files.
- **Group 2: *Asset Link Management/Telemetry Ingestion***: equivalent in complexity to *Prosumer Management*, namely a three-participant message collaboration (`ProsumerForAssetLink` → `AssetLinkManagement` → `TelemetryManagement`), where Operate shows three separate process instances that must be alternately advanced; the same two corrections (Defects 1 and 2) were applied here too.

For *Group 1*, each flow follows the same shape: `POST /v2/process-instances` with the bare `processDefinitionId` (no start variables required), one user task that selects the test subject (a `Grid Cell ID` of `LISBON-DT`, an `Asset ID` of `BATT-001/SOLAR-001`, etc.), then an automatic chain of service tasks that aggregates data from the relevant microservices through Kong and posts the result to the owning microservice’s “emit/publish” endpoint. *Group 2* follows the Initiator/Executor pattern described in Section 2, seeded with `UtilityOperatorID = 1` (`ArcoCegoLisbon`) and `ProsumerID = 1` or `2` (the `João Silva` record created in Section 3.2).

3.5 Automated end-to-end test suite (e2e-tests/)

As a complement to the manual, Operate-driven walkthrough documented in `BPMN/EndToEnd_Tests.md` (and summarised throughout this section), the `e2e-tests/` Maven module translates the same eight flows into an executable JUnit 5 suite, so that the procedure can be re-run against any freshly deployed environment with a single `mvn test` instead of manually clicking through Tasklist.

Every test follows the same three-step shape as the manual run it automates: (i) **start** the process instance through the Zeebe gRPC client (`zeebe-client-java`, the programmatic equivalent of the `POST /v2/process-instances` call of Listing 1); (ii) **drive** it with a `ZeebeClient` job worker subscribed to the `io.camunda.zeebe:userTask` job type, which completes each generated user task with the same seed-data values used in the manual run (`LISBON-DT`, `BATT-001`, `UtilityOperatorID = 1`, `promise/accept` checkboxes, etc.), distinguishing tasks by their BPMN `elementId` exactly as Operate distinguishes them by name; and (iii) **verify persistence through Kong** with REST-assured, asserting that the owning microservice’s endpoint now returns the expected record (`GET /prosumer/Prosumer`, `GET /assetlink/AssetLink`, `GET`

/utility-operator/UtilityOperator, etc. — the same curl verifications already documented in BPMN/EndToEnd_Tests.md). A shared BaseE2ETest class centralises the ZeebeClient and RestAssured setup/teardown, both pointed at the target environment through the zeebe.address and api.gateway.url system properties, so the same suite runs unchanged after a redeploy that produces new EC2 DNS names. The AI Forecasting test additionally polls with Awaitility for the emitted event, since that flow performs a real LLM inference call through Ollama and can take noticeably longer than the others (Section 2, flow 8).

4 Source Code, Infrastructure and Parametrization Documentation

This section documents the artefacts that are *new* in Sprint 2: the BPMN process definitions and the Camunda/Kong/Konga infrastructure that orchestrates and exposes the Sprint 1 microservices. The microservice source code, RDS/Kafka Terraform, deployment automation and Kafka parametrization were delivered in Sprint 1; that documentation is referenced rather than duplicated, and is summarised in Table ?? for completeness.

4.1 Repository layout of the orchestration layer

```

1 EI-Project/
2   +-- BPMN/
3   | +-- ProsumerManagement.bpmn # Flow 1 (Initiator/Executor collab.)
4   | +-- UtilityOperatorManagement.bpmn # Flow 2 (linear)
5   | +-- AssetLinkManagement.bpmn # Flows 3 & 4 (3-participant collab.)
6   | +-- FlexibilityEmission.bpmn # Flow 5 (linear)
7   | +-- GridBalancing.bpmn # Flow 6 (linear)
8   | +-- EnergyAnalytics.bpmn # Flow 7 (linear)
9   | +-- AIForecasting.bpmn # Flow 8 (linear)
10  | +-- EndToEnd_Tests.md # Step-by-step E2E test script (all 8 flows)
11  +-- Camunda-Terraform/
12  | +-- EC2CamundaEngine.tf # t3.large EC2 + security group
13  | +-- deploy.sh # user-data: boots Camunda 8.8.9 stack
14  +-- KongTerraform/
15  | +-- EC2Kong.tf # t3.small EC2 + security group
16  | +-- deploy.sh # user-data: installs/starts Kong
17  | +-- setup-kongGateway.sh # registers 8 services + 8 routes
18  +-- KongaTerraform/ +-- EC2Konga.tf # t3.small EC2 running the Konga admin
    UI

```

Listing 6: Layout of the new orchestration-layer artefacts inside the project repository

4.2 Camunda 8 engine: Terraform

Camunda-Terraform/EC2CamundaEngine.tf provisions a single t3.large aws_instance (Camunda 8 needs more memory than the t3.small tier used for the microservices, since it runs Zeebe, Operate, Tasklist and Elasticsearch together) with an all-open security group (ports 0–65535, 0.0.0.0/0; adequate for a coursework demonstration, as already noted for the microservice security groups in Sprint 1) and a user_data_replace_on_change = true script that pulls and starts the official Camunda 8.8.9 Docker Compose distribution, exposing Operate/Tasklist/the REST API on ports 8080/8081/8082.

```

1 resource "aws_instance" "exampleInstallCamundaEngine" { ami = "ami-07
    ff62358b87c7116" instance_type = "t3.large" vpc_security_group_ids = [
    aws_security_group.instance.id] key_name = "vockey" user_data = "${file("
    deploy.sh")}" user_data_replace_on_change = true
2 }

```

Listing 7: Core resource definition of the Camunda EC2 instance (`EC2CamundaEngine.tf`)

4.3 Kong API Gateway: Infrastructure, Registration and Routing

The Kong implementation acts as the central communication hub of the platform. No microservice is directly exposed to Camunda; all interactions from the orchestration engine pass through this intermediate layer, which guarantees proper routing and architectural isolation.

Provisioning and Installation

The `KongTerraform/EC2Kong.tf` file provisions the instance (`t3.small`, with an all-open security group pattern) and exports its public DNS as a Terraform output (`address`). The installation is handled by the `deploy.sh` script, injected as `user_data` during boot, which installs and starts the Kong gateway. Kong exposes port 8000 (the proxy that receives requests from Camunda) and port 8001 (the Admin API, used to receive local configuration commands).

```
1 resource "aws_instance" "exampleInstallKong" { ami = "ami-07ff62358b87c7116"
      instance_type = "t3.small" vpc_security_group_ids = [aws_security_group.
      instance.id] key_name = "vockey" user_data = "${file("deploy.sh")}"
2 }
3
4 output "address" { value = aws_instance.exampleInstallKong.public_dns
      description = "Connect to the KONG at this endpoint"
5 }
```

Listing 8: Core resource definition of the Kong EC2 instance (`EC2Kong.tf`)

Service Registration and Routing Automation

The `setup-kongGateway.sh` script is responsible for transforming the bare Kong instance into a fully functional API Gateway. It receives the Kong DNS and the DNS of every microservice as positional parameters. It polls the Admin API (`http://<kong-dns>:8001/status`) until it responds (up to 60 attempts), and then uses a `register_service` helper to make HTTP POST calls to create the entities. For each microservice, it registers a **Service** (pointing to `http://<microservice-address>:8080`) and associates a **Route** corresponding to its domain path.

```
1 register_service { local service_name=$1 service_url=$2 route_path=$3
      route_name=$4 curl -s -X POST "http://$addressKong:8001/services/" \ --
      data "name=$service_name" --data "url=$service_url" \ -H "Content-Type:
      application/x-www-form-urlencoded" curl -s -X POST "http://$addressKong
      :8001/services/$service_name/routes" \ --data "name=$route_name" --data "
      paths=$route_path" \ --data "strip_path=true" \ -H "Content-Type:
      application/x-www-form-urlencoded"
2 }
3
4 # e.g.
5 register_service "prosumer-service" "http://$addressProsumer:8080" "/"
      prosumer" "prosumer-route"
6 # ... one call per microservice, see Table 1
```

Listing 9: Service/route registration pattern used for every microservice in `setup-kongGateway.sh`

The `strip_path` Option and BPMN Integration

Two technical details are critical for the orchestration layer to function properly:

1. **The `strip_path=true` option:** Injected when creating the routes in Kong. Without it, a request to `POST /prosumer/Prosumer` (via Kong) would be forwarded to the upstream microservice with the exact same path, resulting in a 404 error. With `strip_path=true`, Kong dynamically removes the route prefix (`/prosumer`) and forwards only `/Prosumer` to the correct JAX-RS endpoint on the target EC2 instance.
2. **Direct BPMN Injection:** Since the endpoint addresses are dynamic with each Terraform deployment, the HTTP Connectors in the BPMN diagrams initially contained a malformed placeholder (e.g., `http://:8000`) resulting from the templating step. This required a global `sed` substitution (as detailed in Section 3.3, Listing 3) to replace the broken targets with the live Kong DNS across all `.bpmn` files before executing the flows.

4.4 Konga administration UI

`KongaTerraform/EC2Konga.tf` follows the same `t3.small` / `all-open-security-group` / `user_data` pattern and starts the Konga container, which was used throughout development to visually confirm that `setup-kongGateway.sh` had registered the expected eight services and eight routes (Table 1) and to inspect/debug routing while the BPMN service tasks were being wired up.

4.5 BPMN process definitions: parametrization conventions

Two parametrization conventions run through every BPMN file and are the most important thing to know when redeploying the orchestration layer against a *new* environment (a fresh `terraform` apply always produces new EC2 public DNS names):

1. **`camundaBaseUrl` process variable:** every Tasklist form-generation service task needs to know the base URL of the Camunda REST API it is running against. It must be supplied when *starting* a none-start-event process (`"variables": {"camundaBaseUrl": "http://<camunda-dns>:8080"}`) and explicitly *forwarded* inside the `variables` object of every message that starts a collaborating process (Defect 1, Section 3.3).
2. **Kong base URL in HTTP Connector service-task targets:** every outbound REST call embedded in a service task's URL must point at `http://<kong-dns>:8000/<route-path>/...` (Table 1), *not* at an individual microservice's EC2 address. After a redeploy that produces a new Kong DNS, all such URLs across the seven `.bpmn` files must be updated, which is exactly what the global `sed` substitution in Listing 3 automates.

Both conventions, the current live endpoint values, and the redeploy command (Listing 4) are kept up to date in `BPMN/EndToEnd_Tests.md`, which doubles as the parametrization runbook for whoever re-runs the demo after a fresh deployment.

5 Conclusion

Sprint 2 closes the gap between the data plane delivered in Sprint 1 and the business-process layer required by the VPPaaS brief. All eight required business processes (*Prosumer Management*, *Utility Operator Management*, *Asset Link Management*, *Telemetry Ingestion*, *Flexibility Emission*, *Grid Balancing Recommendation*, *Energy Analytics* and *Flexibility Forecasting (AI)*) were modelled in BPMN, deployed to Camunda 8, and wired to the underlying microservices exclusively through the Kong API Gateway, following the single-entry-point integration pattern the brief requires.

Critically, the flows were not only modelled but **exercised end to end** against the live AWS deployment: the *Prosumer Management* flow was run from a cold Camunda REST API call all the way to a persisted row in the shared RDS database (Section 3.2), and the remaining seven flows were documented with reproducible start commands, Tasklist step sequences and Kong-side

verification calls (`BPMN/EndToEnd_Tests.md`). That process surfaced three classes of latent defects (a missing cross-process variable, a systemic placeholder left unresolved in 21 service-task URLs across all seven BPMN files, and a variable-name case mismatch silently dropping a business field), all of which were corrected at the BPMN-definition level and re-validated with fresh process instances (Section 3.3). Finding and fixing these issues is, in itself, the strongest evidence that the integration layer was genuinely tested rather than merely modelled: none of the three defects was visible from a static reading of the diagrams, and all three would have caused the live demonstration to fail had they not been caught beforehand.

Finally, this report documents, alongside the new orchestration-layer artefacts (BPMN models, Camunda/Kong/Konga Terraform, Kong service-registration script, parametrization conventions for `camundaBaseUrl` and the Kong base URL), the full installation procedure that takes the platform from zero to a running, end-to-end-testable system, building on and extending the Sprint 1 documentation.